

条款24 区分通用引用和右值引用

通用引用: 既可以表示左值引用又可以表示右值引用

① 模板的通用引用 要求: $T \&\&$ + 类型推导

```
template<class T>
```

```
void func(T &&param) {...}
```

```
int a = 0; func(a);           int&
```

```
func(10);                     int
```

```
func<int>(10);                int
```

→ 指定类型, 传参只能

为右值引用

```
func<int>(a);                 int 空句报错
```

```
template<class T>
```

```
void func(vector<T> &&param);
```

右值引用

形式不满足 $T \&\&$ auto&&

```
template<class T>
```

```
void func(const T &&param) {...}
```

右值引用, 与上同理

```
template<class T, class Allocator = allocator<T>>
```

```
class vector { public push_back(T&&x); }
```

x 仍是右值引用 为什么? $\Rightarrow$ 实例化 `vector` 时

T 类型已经推导决定了, `push_back` 只有在 `vector`

实例化才会存在

② 可变参数模板的通用引用 要求:

$Args \&\&... + \text{类型推导}$

```
template<class T>
```

```
class Vector{public: template<... Args>
```

```
void emplace_back( $Args \&\&... args$ );}
```

可变参数模板的通用引用

③ auto 的通用引用 要求: $auto \&\& / auto \&\&... + \text{类型推导}$

常见于 lambda 表达式

```
auto timeFuncInvocation = [](auto && func,  
auto && params){
```

```
    struct timer;
```

```
    std::forward<decltype(func) (&& func)
```

```
        std::forward<decltype(params)...>(&& params);
```

```
    ... };
```